MBU DIGITAL LIBRARY · PRACTICE LEARNING RESOURCE

Java Programming

JAVA101 | Suggested sequence · Semester 2 | Student-authored study guide

How to use these notes

Java syntax, object-oriented design, collections, exceptions, and small testable programs. Work through the units in order, reproduce each example, then answer the review prompts from memory. This is supplemental practice material, not an official MBU syllabus or assessment. Course sequencing is suggested and should be checked against your current approved syllabus.

MODULE 01 · Types, methods, and control flow

Java variables have declared types. Methods specify return types and parameters; static belongs to a class, while instance members belong to objects. Use braces and meaningful names, and validate input boundaries.

Worked example

```
static double average(int total, int count) { if (count <= 0) throw new
IllegalArgumentException("count"); return (double) total / count; }
```

Check your understanding

- Why cast before dividing integer values?
- What is the difference between a class and an object?

MODULE 02 · Classes, interfaces, and exceptions

Keep fields private and expose behavior through methods. Interfaces specify capabilities. Use exceptions for exceptional failures and make resource lifetime explicit with try-with-resources.

Worked example

```
try (var reader = Files.newBufferedReader(path)) { String firstLine = reader.readLine(); }
```

Check your understanding

- Why encapsulate mutable fields?
- What does try-with-resources guarantee?

MODULE 03 · Collections, generics, and tests

List models ordered values; Set models uniqueness; Map associates keys with values. Generics preserve type information. Test expected behavior, empty input, boundary values, and failure cases.

Worked example

```
Map<String, Integer> counts = new HashMap<>(); counts.merge("data", 1, Integer::sum);
```

Check your understanding

- What problem do generics solve?
- When would a Set be more appropriate than a List?

Revision checklist

- %¡ Can I explain the main idea without copying the notes?
- %¡ Can I reproduce the example and state its assumptions?
- %¡ Can I test an edge case and describe what should happen?
- %¡ Have I recorded a limitation and checked a primary reference?

Further reading

Use the official documentation and open references linked in the portal. Check the publication/version and license at the source before redistribution. Cite the source and access date in coursework.